



摘要

分子动力学 (Molecular Dynamics, MD) 模拟是研究生物大分子动态行为的重要计算工具, 被誉为“计算显微镜”。随着全细胞模拟目标的提出, 传统基于快速傅里叶变换 (Fast Fourier Transform, FFT) 的长程静电求解方法——如粒子网格 Ewald (Particle Mesh Ewald, PME) 和粒子-粒子-粒子-网格 (Particle-Particle Particle-Mesh, PPPM) ——在大规模并行计算中面临严重的通信瓶颈。本研究针对这一关键问题, 提出了一种基于高斯和分解 (Sum-of-Gaussians, SOG) 与随机批量采样 (Random Batch Sampling) 相结合的新型长程静电求解框架, 即基于随机批量采样与高斯和分解的长程静电方法 (Random Batch Sum-of-Gaussians, RBSOG)。

本研究构建了面向膜体系的分子动力学模拟原型系统, 实现了三种长程静电求解器: 直接求和参考解 ($O(N^2)$)、PPPM 基线方法和 RBSOG 方法。通过系统化的基准测试, 从计算性能、统计方差和结构稳定性三个维度对 PPPM 和 RBSOG 进行了全面比较。基于本文重新串行执行的真实实验结果, RBSOG 在当前纯 Python 原型中的平均每步时间显著高于 PPPM, 在 10 个随机种子统计下约为其 35.868 倍; 同时, RBSOG 的压力方差均值也高于 PPPM, 说明该方法在当前实现阶段尚未展现出统计稳定性优势。不过, 批量大小扫描与 JIT 消融实验表明, RBSOG 对实现细节和参数选择较为敏感, 且 JIT 加速可带来约 4.271 倍的内核级提速, 这表明该方法仍具有进一步工程优化空间。

本研究的主要贡献包括: (1) 提出了 RBSOG 长程静电求解框架, 将高斯和分解与随机批量采样相结合, 为大规模分子动力学模拟提供了新的算法思路; (2) 建立了兼顾计算效率、统计方差和结构稳定性的综合评测体系; (3) 构建了可复现的自动化实验与论文生成流程; (4) 通过系统化实验分析, 明确给出了当前原型阶段 RBSOG 与 PPPM 之间的真实性能差距与优化方向。研究结果表明, RBSOG 方法在当前实现下尚未优于 PPPM, 但具有进一步工程优化和向更大规模体系迁移的潜力, 为未来全细胞分子动力学模拟中的长程静电求解提供了新的技术路径。

关键词: 分子动力学模拟, 长程静电, 高斯和分解, 随机批量采样, 全细胞模拟, 膜体系



Abstract

Molecular Dynamics (MD) simulation is an important computational tool for studying the dynamic behavior of biological macromolecules, known as the “computational microscope.” With the advent of whole-cell simulation goals, traditional long-range electrostatics solvers based on the Fast Fourier Transform (FFT)—such as Particle Mesh Ewald (PME) and Particle-Particle Particle-Mesh (PPPM)—face severe communication bottlenecks in large-scale parallel computing. This study addresses this critical problem by proposing a novel long-range electrostatics framework, Random Batch Sum-of-Gaussians (RBSOG), which combines Sum-of-Gaussians (SOG) decomposition with random batch sampling.

We built a membrane-oriented molecular dynamics prototype with three long-range electrostatics solvers: direct pairwise Coulomb reference ($O(N^2)$), the PPPM baseline, and the proposed RBSOG method. Through systematic benchmarking, we compared PPPM and RBSOG across computational performance, statistical variance, and structural stability. Based on the final serially executed experiments in this thesis, RBSOG remains about 35.868 times slower than PPPM in the current pure-Python prototype and also shows higher pressure variance in the 10-seed benchmark. Nevertheless, batch-size sweep and just-in-time compilation ablation results indicate that the method is highly sensitive to implementation details and still has room for optimization. Furthermore, JIT acceleration provides about 4.271 times kernel-level speedup in the current implementation, and we introduced a weighted Pareto frontier analysis method for automated batch size parameter P recommendation.

The main contributions of this study include: (1) proposing the RBSOG long-range electrostatics framework combining SOG decomposition with random batch sampling, providing a new algorithmic approach for large-scale MD simulation; (2) establishing a comprehensive evaluation system balancing computational efficiency, statistical variance, and structural stability; (3) constructing a reproducible automated experiment and paper generation pipeline; (4) identifying, through real benchmark evidence, the current performance gap and optimization priorities of the prototype. The results indicate that RBSOG has potential for further engineering optimization and migration to larger systems, provid-



ing a new technical pathway for long-range electrostatics in future whole-cell molecular dynamics simulations.

Keywords: Molecular Dynamics, Long-Range Electrostatics, Sum-of-Gaussians, Random Batch Sampling, Whole-Cell Simulation, Membrane System



目 录

第一章 引言	1
1.1 研究背景与意义	1
1.2 长程静电求解的挑战与机遇	2
1.3 研究目标与内容	3
第二章 方法	5
2.1 研究目标与总体流程	5
2.2 代理膜体系与动力学框架	5
2.3 三类长程静电求解器	6
2.4 实验套件设计	8
2.5 统计方法	10
2.6 自动化资产生成	11
第三章 结果	12
3.1 实验套件总览	12
3.2 PPPM 网格公平性扫描	12
3.3 10 种子主基准测试	13
3.4 RBSOG 批量大小扫描	14
3.5 Direct Reference 精度验证	16
3.6 规模扫描实验	18
3.7 长轨迹稳定性实验	20
3.8 RBSOG 参数消融与 Profiling	23
3.9 稳健统计分析	26
3.10 本章小结	27
第四章 讨论	28
4.1 主要结果的含义	28
4.2 误差来源与参数敏感性	29
4.3 Profiling 揭示的优化方向	30
4.4 与相关工作的关系	30
4.5 局限性	31
4.6 后续工作	31



第五章 结论	32
5.1 主要工作总结	32
5.2 核心实验结论	32
5.3 创新点与意义	33
5.4 最终判断	33
附录 A 实现补充与复现实验说明	34
A.1 真实实现说明	34
A.2 RBSOG 求解器真实核心代码节选	34
A.3 近邻表与短程求解真实实现节选	37
A.4 SOG 核拟合真实实现节选	38
A.5 复现实验命令	39
A.6 附录小结	41
参考文献	42
致谢	43



第一章 引言

1.1 研究背景与意义

计算显微镜：跨越时空尺度的生物学新范式

在过去半个世纪中，结构生物学经历了从 X 射线晶体学到冷冻电子显微镜 (Cryogenic Electron Microscopy, Cryo-EM) 的革命性飞跃，使得人类能够以原子级分辨率解析单个蛋白质及其复合物的静态结构。然而，生命现象的本质在于”动态”与”相互作用”。细胞内部并非稀溶液环境，而是一个拥挤、异质且高度动态的软物质系统。大分子拥挤效应 (Macromolecular Crowding) 导致细胞内蛋白质的扩散行为、构象稳定性以及酶促反应动力学与体外稀溶液环境存在显著差异^[1]。传统的实验手段难以在保持细胞完整性的同时，以原子分辨率捕捉毫秒级至秒级的动态过程。

在此背景下，分子动力学 (Molecular Dynamics, MD) 模拟作为一种”计算显微镜” (Computational Microscope)，填补了实验观测在时空分辨率上的空白^[2]。基于牛顿经典力学方程，MD 模拟能够重现生物大分子在力场驱动下的运动轨迹。随着高性能计算 (High Performance Computing, HPC) 硬件——特别是图形处理器 (Graphics Processing Unit, GPU) 和百亿亿次 (Exascale) 超级计算机——的迅猛发展，以及增强采样算法和粗粒化 (Coarse-Grained, CG) 力场的成熟，MD 模拟的研究对象已从单一蛋白质扩展至病毒衣壳 (如 HIV-1, SARS-CoV-2)、细胞器 (如光合色素体) 乃至全细胞 (Whole-Cell) 尺度^[3]。

全细胞 MD 模拟不仅是计算生物学的终极挑战之一，更是连接分子生物学与系统生物学的桥梁。它旨在构建一个包含所有基因产物、代谢物、膜结构及溶剂环境的”数字孪生细胞”，从而在第一性原理 (First Principles) 层面上理解生命的涌现性 (Emergence) 特征^[4]。

科学意义：解构生命物质的物理本质

开展全细胞尺度的分子动力学模拟研究具有深远的科学意义和应用价值：

阐明拥挤环境下的物理化学机制：细胞质中的大分子浓度高达 200-400 g/L，占据了约 30% 的体积。这种极端的拥挤环境产生的排斥体积效应 (Excluded Volume Effect) 和非特异性相互作用 (Quinary Structure) 深刻影响着蛋白质的折叠、组装及相分离 (Phase Separation) 行为。全细胞模拟能够量化这些效应，揭示细



胞如何利用拥挤环境调控生化反应速率^[5]。

整合多组学数据并验证生物学假设：全细胞模型的构建过程本身就是对基因组学、蛋白质组学、脂质组学及代谢组学数据的一次深度整合与自洽性验证。通过模拟，可以发现现有实验数据的缺失环节（如未知功能的基因产物），并预测生物大分子在细胞内的空间分布规律^[4]。

指导合成生物学与精准药物设计：以最小细胞（如 JCVI-syn3A）为对象的模拟有助于确定维持生命所需的最小基因集，为人造细胞的设计提供蓝图。同时，全病毒或全细胞模拟能够揭示药物分子在复杂生物环境中的输运路径及与靶点的动态结合机制（如 HIV-1 衣壳抑制剂 Lenacapavir 的作用机理），为开发新型抗病毒药物提供理论依据^[3]。

推动计算科学的发展：全细胞模拟涉及数亿至数十亿粒子的计算，对超级计算机的并行效率、数据存储及可视化技术提出了极限挑战，将直接推动 Exascale 计算技术和大数据分析算法的革新^[4]。

领域现状：通往全细胞模拟的里程碑

近年来，该领域取得了一系列标志性成果，为全细胞模拟奠定了坚实基础：

病毒衣壳的全原子模拟：2013 年，UIUC 团队实现了 HIV-1 成熟衣壳（约 6400 万原子）的全原子模拟，揭示了衣壳的力学性质和离子通透性^[6]。随后，SARS-CoV-2 病毒包膜（约 3.05 亿原子）的模拟揭示了刺突蛋白的糖基化屏蔽机制和动态构象^[7]。

细菌细胞质的拥挤模拟：RIKEN 和密歇根州立大学利用 GENESIS 软件实现了大肠杆菌细胞质的大规模全原子模拟，发现代谢物与蛋白质的相互作用会改变蛋白质的表面扩散行为^[4]。

最小细胞 JCVI-syn3A 的初步尝试：J. Craig Venter 研究所（J. Craig Venter Institute, JCVI）与 UIUC 合作，构建了 JCVI-syn3A 的马蒂尼（Martini）粗粒化模型（约 6 亿粒子），并结合反应-扩散主方程（Reaction-Diffusion Master Equation, RDME）实现了首个包含完整代谢和遗传信息处理过程的全细胞动力学模型^[3]。

1.2 长程静电求解的挑战与机遇

传统方法的通信瓶颈

在分子动力学模拟中，长程静电相互作用的计算是最为关键的性能瓶颈之一。传统的 Ewald 求和方法及其网格加速版本——粒子网格 Ewald（Particle Mesh



Ewald, PME)^[8] 和粒子-粒子-网格 (Particle-Particle Particle-Mesh, PPPM)^[9]——通过将长程相互作用分解为短程和长程两个部分, 实现了 $O(N \log N)$ 的计算复杂度。然而, 这些方法依赖于全局快速傅里叶变换 (Fast Fourier Transform, FFT), 在大规模并行计算中面临严重的通信瓶颈。

具体而言, FFT 要求在每个时间步进行全局数据交换, 这在分布式内存架构的超级计算机上会产生显著的通信开销。随着体系规模的增大和处理器数量的增加, 通信时间可能超过计算时间, 导致并行效率急剧下降。这一问题在全细胞模拟场景中尤为突出, 因为全细胞体系的粒子数可能达到数十亿量级^[4]。

随机批量方法的兴起

近年来, 随机批量方法 (Random Batch Methods, RBM) 为大规模粒子系统模拟提供了新的思路。Jin 等人提出的随机批量相互作用粒子系统方法^[10], 通过随机采样策略将 $O(N^2)$ 的相互作用计算降低为 $O(N)$, 同时保持了统计无偏性。这一方法已被成功应用于分子动力学、朗之万动力学和 Vlasov 方程等领域。

随机批量方法的核心思想是: 在每个时间步, 随机选择一部分相互作用进行计算, 而非计算所有相互作用。通过精心设计的采样策略和适当的统计校正, 可以在保持数值稳定性的前提下显著降低计算成本。更重要的是, 随机批量方法天然支持并行化, 因为它避免了全局通信的需求^[10]。

高斯和分解的数值优势

高斯和分解 (Sum-of-Gaussians, SOG) 是一种将复杂函数表示为多个高斯函数加权和技术。在量子力学中, SOG 张量神经网络已被成功应用于高维薛定谔方程的求解^[11]。SOG 分解的优势在于:

(1) 高斯函数具有良好的解析性质, 便于数值计算; (2) 高斯函数的可分离性使得高维积分可以分解为低维积分的乘积; (3) 高斯函数的快速衰减性保证了数值稳定性; (4) SOG 分解可以与随机采样策略自然结合。

将 SOG 分解应用于长程静电相互作用的计算, 有望实现更平滑的数值行为和更好的并行扩展性。

1.3 研究目标与内容

核心研究目标

本研究的核心目标是探索一种兼顾数值平滑性、统计稳定性与算法扩展潜力的新型长程相互作用处理框架, 并以基于随机批量采样与高斯和分解的长程



静电方法 (Random Batch Sum-of-Gaussians, RBSOG) 作为当前阶段的核心研究对象。具体目标包括:

(1) 构建面向膜体系的分子动力学模拟原型系统, 实现 RBSOG 方法的原型验证; (2) 建立系统化的基准测试体系, 从计算性能、统计方差和结构稳定性三个维度评估 RBSOG 方法; (3) 通过实验分析揭示 RBSOG 方法的优势与局限, 为后续工程优化提供依据; (4) 建立可复现的自动化实验与论文生成流程, 为后续研究奠定基础。

论文组织结构

本文的组织结构如下:

第一章为引言, 介绍研究背景、科学意义、领域现状以及长程静电求解的挑战与机遇。

第二章为方法, 详细介绍分子动力学基础理论、RBSOG 算法框架、膜体系构建方法以及评测指标设计。

第三章为结果, 展示 PPPM 与 RBSOG 的基准测试结果、批量大小扫描实验、结构稳定性分析以及 JIT 消融实验。

第四章为讨论, 对实验结果进行深入分析, 讨论 RBSOG 方法的优势与局限, 以及与现有工作的比较。

第五章为结论, 总结本研究的主要贡献和创新点, 展望未来研究方向。



第二章 方法

2.1 研究目标与总体流程

本文关注的问题不是在成熟分子动力学引擎中复现完整生产级工作流，而是在一个可控、可复现的研究原型中回答三个更具体的问题：

(1) 在统一的代理膜体系下，当前实现的 RBSOG 与 PPPM 相比，运行时间和统计波动表现如何；

(2) 在可以计算 direct reference 的小规模体系上，RBSOG 的误差是否具有可调节性；

(3) 如果当前实现尚未优于 PPPM，那么瓶颈主要位于哪些实现阶段，后续是否存在明确的优化方向。

围绕这三个问题，本文将实验流程组织为一条统一的数据管线：首先由 `python -m rbsog_md.studies --out-root results/thesis_suite` 生成完整实验套件；随后由 `docs/report/generate_thesis_assets.py` 把结果目录汇总为 `thesis_template/generated/` 中的表格、图像和宏定义；最后通过 `thesis_template/build_publication.py` 进行公开安全版论文编译。这样做的目的，是让章节中的关键数字、图表和实验目录之间保持单一来源，降低手工拷贝带来的不一致风险。

2.2 代理膜体系与动力学框架

代理膜体系

考虑到本文的重点是长程静电算法比较而非全原子力场验证，体系构建采用膜启发的粗粒化代理模型。体系由两部分组成：前 `n_lipids` 个粒子表示双层膜中的脂质代理粒子，后 `n_solvent` 个粒子表示显式溶剂代理粒子。脂质粒子沿 z 方向分成上下两个 leaflet，并在 x - y 平面上以带扰动的网格方式初始化；溶剂粒子则在整个盒子内均匀随机分布。该设计保留了本文最关心的三个要素：

- (1) 长程库仑相互作用仍然主导求解器成本；
- (2) 体系保留显式粒子统计涨落，从而能够观察压力方差和温度控制行为；
- (3) 双层膜几何结构仍然允许导出面积/脂质和膜厚代理量等结构稳定性指标。

主基准体系固定为 128 个脂质粒子和 512 个溶剂粒子，总粒子数为 640，模拟盒子大小为 (16, 16, 20) 约化单位。针对 direct reference 精度实验，本文还构



造了 $N = 160, 320, 640$ 的小规模体系；针对经验复杂度扫描，则进一步扩展到 $N = 960$ 和 $N = 1280$ 。

积分与系综控制

系统时间推进采用 Velocity Verlet 算法。若记粒子位置、速度、质量和作用力分别为 $\mathbf{r}_i$ 、 $\mathbf{v}_i$ 、 m_i 和 $\mathbf{F}_i$ ，则更新规则可写为：

$$\mathbf{r}_i(t + \Delta t) = \mathbf{r}_i(t) + \mathbf{v}_i(t)\Delta t + \frac{1}{2m_i}\mathbf{F}_i(t)\Delta t^2, \quad (2.1)$$

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t) + \frac{\mathbf{F}_i(t) + \mathbf{F}_i(t + \Delta t)}{2m_i}\Delta t. \quad (2.2)$$

温度和压力控制采用 Berendsen thermostat 与 Berendsen barostat。它们并不追求严格生产级采样，而是用于在统一条件下比较不同长程求解器的相对行为。瞬时压力由 virial 公式估计：

$$P = \frac{Nk_B T + W/3}{V}, \quad (2.3)$$

其中 N 为粒子数， T 为瞬时温度， W 为 virial， V 为体系体积。所有实验均使用约化单位，这意味着本文讨论的是算法工程层面的相对结论，而非特定生物体系的物理可比量纲。

结构代理量定义

本文使用两个结构代理量追踪膜体系的几何稳定性。面积/脂质定义为

$$A_{\text{lipid}} = \frac{L_x L_y}{N_{\text{lipids}}/2}, \quad (2.4)$$

其中分母使用单层 leaflet 中的脂质数，因此能够反映双层膜在横向方向上的平均扩展程度。膜厚代理量则由上下 leaflet 脂质粒子在 z 方向的均值差给出，用于近似表示法向结构变化。对这两个量，本文既考察整体标准差，也考察随时间的线性漂移率与滚动均值曲线。

2.3 三类长程静电求解器

Direct Reference

direct solver 直接对所有粒子对进行库仑求和，其势能表达式为



$$U_{\text{direct}} = \sum_{i < j} \frac{q_i q_j}{4\pi\epsilon_0 r_{ij}}. \quad (2.5)$$

该方法的计算复杂度为 $O(N^2)$ ，不适合在主基准和长轨迹实验中使用，但在小规模体系上可以作为误差参考。本文在 $N = 160, 320, 640$ 三组小体系上使用 direct solver 与 PPPM、RBSOG 分别比较，用于衡量相对力误差、势能误差、virial 误差和压力误差。

PPPM 基线求解器

PPPM 求解器保留经典的“粒子-网格-粒子”工作流：先把电荷沉积到规则网格，再在频域中完成 Poisson 求解，最后把电场插值回粒子位置。该路径在成熟实现中通常具有较高效率，但会依赖 FFT 这一全局变换步骤^[8,9]。在当前原型中，PPPM 的 profile 诊断被拆分为三段时间：电荷沉积、FFT 求解和场插值，这为后续 profiling 分析提供了直接证据。

本文没有把 PPPM 网格固定为传统经验值，而是先进行一轮 **PPPM 公平性扫描**：候选网格取 $16^3, 24^3, 32^3, 48^3, 64^3$ ，同时考察运行时间和 direct force RMSE，再用归一化乌托邦距离选择后续主基准使用的网格。该设计的目的，是避免在 RBSOG 尚处于原型阶段时，直接拿一个明显偏向高精度而非公平成本的 PPPM 配置作为唯一比较对象。

RBSOG 求解器

RBSOG (Random Batch Sum-of-Gaussians) 把长程静电计算分为短程和长程两部分：短程部分由近邻表求解器负责，长程部分则通过高斯和分解与随机批量采样估计。若把库仑核近似表示为

$$\frac{1}{r} \approx \sum_{m=1}^M w_m \exp(-\alpha_m r^2), \quad (2.6)$$

则长程项可以被写成多个平滑高斯核的加权和，从而适合以随机采样方式近似。当前实现不是简单地从粒子对集合中均匀抽样，而是先根据粒子电荷绝对值构造重要性分布，再独立采样有序粒子对；对于有效样本 (i, j) ，其统计权重为

$$w_{ij}^{\text{sample}} = \frac{1}{m_{\text{samples}} p_i p_j}, \quad (2.7)$$



其中 $m_{\text{samples}} \approx P \cdot N$ 为总采样数, p_i 和 p_j 为粒子级采样概率。批量大小 P 决定了长程采样的规模, SOG 项数 M 控制高斯核近似的自由度, 短程截断半径 r_c 决定了短程与长程的分工边界。这三个量共同决定 RBSOG 的时间-方差-误差平衡^[10,11]。

近邻表与 JIT 加速

RBSOG 短程部分使用 Verlet 近邻表, 并记录 pair-list 构建时间与短程核计算时间。若启用 Numba, 短程近邻核和 RBSOG 长程核都可进入 JIT 路径。本文专门进行了 JIT 消融实验和 profiling 实验, 用于回答“当前性能差距究竟是算法层瓶颈还是解释器层瓶颈”这一问题。

2.4 实验套件设计

完整实验套件由七类互补实验组成, 分别对应不同层次的问题。

统一配置

主实验与扩展实验的统一配置如表2.1 所示。与旧版本论文不同, 本文不再以单一三种子基准作为所有结论的唯一来源, 而是把主基准、精度参考、规模扫描、长轨迹、消融和 profiling 放入同一套自动化流程中。

表 2.1 Thesis 套件的核心配置摘要

Parameter	Value
Primary system	lipids=128, solvent=512
Simulation box	(16, 16, 20) reduced units
Main benchmark	1000 steps, sample interval 20, 10 seeds
PPPM grid fairness choice	16^3
Recommended RBSOG batch	$P = 100$
Short-range cutoff	1.2 reduced units
Default SOG terms	$M = 12$
Neighbor list	skin=0.3, rebuild interval=10
Long stability study	5000 steps, sample interval 50, 5 seeds
Scaling study	$N = 320, 640, 960, 1280$ with 3 seeds each



PPPM 公平性扫描

该实验在统一主体系上比较不同网格尺寸的 PPPM 运行时间与 direct force RMSE。若记网格配置为 g ，运行时间和误差分别为 $t(g)$ 与 $e(g)$ ，归一化后得到 $\hat{t}(g)$ 与 $\hat{e}(g)$ ，则公平性评分定义为

$$\text{score}_{\text{fair}}(g) = \sqrt{\hat{t}(g)^2 + \hat{e}(g)^2}. \quad (2.8)$$

分数最低的网格被用于主基准、长轨迹与规模扫描中的 PPPM 基线。该步骤的作用，是把“PPPM 基线”从经验固定超参数，转变为一个由同一套数据驱动选出的配置。

10 种子主基准与批量扫描

主基准在 $N = 640$ 体系上比较 PPPM 和 RBSOG，随机种子为 42 至 51，共 10 次独立运行。对于 RBSOG，本文另外执行 $P \in \{50, 100, 200, 400\}$ 的批量扫描，既报告相对 PPPM 的速度比和压力方差比，也保留 Pareto 前沿上的乌托邦距离用于自动推荐。

若目标函数对运行时间和压力方差的权重分别为 w_t 和 w_v ，则批量推荐分数写作

$$\text{score}(P) = \sqrt{w_t \hat{t}(P)^2 + w_v \hat{v}(P)^2}, \quad (2.9)$$

其中 $\hat{t}(P)$ 和 $\hat{v}(P)$ 是按候选集归一化后的时间与方差。本文采用 $w_t : w_v = 2 : 1$ ，即更偏向运行时间，但只在 RBSOG 候选集中推荐，不把“推荐”误写成“已经优于 PPPM”。

Direct Reference 精度实验

为了把“速度差距”和“数值误差”分离开来，本文在 $N = 160, 320, 640$ 三组小规模体系上进行 direct reference 比较。每个快照先用 PPPM 短暂平衡，再分别使用 direct、PPPM 和多组 RBSOG 参数计算静电力、势能和 virial。核心指标包括相对力 RMSE：

$$\text{RMSE}_F^{\text{rel}} = \sqrt{\frac{\sum_i \|\mathbf{F}_i^{\text{approx}} - \mathbf{F}_i^{\text{ref}}\|_2^2}{\sum_i \|\mathbf{F}_i^{\text{ref}}\|_2^2}}, \quad (2.10)$$

以及势能、virial 和压力的相对标量误差：



$$\varepsilon_x^{\text{rel}} = \frac{|x_{\text{approx}} - x_{\text{ref}}|}{\max(|x_{\text{ref}}|, 10^{-12})}. \quad (2.11)$$

这一实验用来回答两个问题：一是公平性选出的 PPPM 网格在小体系上究竟有多粗；二是 RBSOG 的 P 与 M 是否确实构成可调节的误差控制旋钮。

经验复杂度、长轨迹与 profiling 实验

规模扫描实验在 $N = 320, 640, 960, 1280$ 上比较 PPPM 与 RBSOG 的平均每步时间，并用对数-对数线性拟合估计经验复杂度指数：

$$\log t = \alpha \log N + \beta. \quad (2.12)$$

其中 α 是经验复杂度斜率。长轨迹稳定性实验则固定主体系，在 5 个随机种子下运行 5000 步，重点观察面积/脂质、膜厚代理量和压力的滚动均值曲线。本文使用长度为 10 个采样点的滑动窗口计算滚动均值和滚动标准差，以减少单点尖峰对结构趋势解释的干扰。

profiling 实验在 PPPM 和 RBSOG 上分别记录每个采样步的诊断量，并把不同阶段的平均耗时除以对应求解器的总耗时，得到

$$s_k = \frac{\bar{t}_k}{\bar{t}_{\text{total}}}, \quad (2.13)$$

其中 $\bar{t}_k$ 表示组件 k 的平均耗时占比。这个定义使得 profiling 结果可以直接回答“下一步优化应该先改哪里”。

2.5 统计方法

均值、标准差与置信区间

对多随机种子主基准，本文首先报告样本均值与样本标准差，并使用 t 分布给出步时间均值的 95% 置信区间半宽。这样做的原因，是步时间属于每次运行都稳定可观测的量，适合用经典区间估计描述其不确定性。

Bootstrap 比例区间

为了直接比较两种求解器的相对差异，本文还对“RBSOG/PPPM”的均值比进行 bootstrap 重采样。相较于只报告两个均值和标准差，这种做法更适合回答“RBSOG 到底慢了多少倍”以及“压力方差平均大了多少倍”。



置换检验与离散度检验

对主基准中的步时间和压力方差，本文使用两类置换检验：

- (1) 均值差置换检验，用于判断两组样本均值差异是否显著；
- (2) 离散度置换检验，用于比较两组样本偏离中位数的平均绝对偏差，从而衡量“种子敏感性是否更强”。

在当前原型中，这一步很重要，因为 RBSOG 的若干统计量并不只体现为均值偏移，还体现为更强的跨种子离散性。

2.6 自动化资产生成

本文的论文图表并不是手工插图，而是由实验输出自动生成。`generate_thesis_assets.py` 读取 `results/thesis_suite/` 中的 CSV、JSON 和 PNG 产物，生成两类内容：

- (1) 供正文直接 `\input` 的 `*.tex` 表格；
- (2) 供正文直接插图的 `*.png` 图像，以及若干在摘要和章节中复用的宏定义。

这种设计意味着，只要重新运行实验套件并重新生成资产，文中的关键结论就会随数据同步更新。对当前毕业论文场景而言，这一点比“单次跑出一个好看的结果”更重要，因为它保证了从实验目录到最终 PDF 之间的追溯链条始终存在。



第三章 结果

3.1 实验套件总览

本章结果均来自 `results/thesis_suite/` 中的重新执行实验。与前一版只围绕少量主基准和批量扫描展开不同，本章将 PPPM 公平性扫描、10 种子主基准、direct reference 精度验证、规模扫描、长轨迹稳定性、参数消融、profiling 和稳健统计统一纳入同一套自动化资产链路。所有表格和图像均由 `docs/report/generate_thesis_assets.py` 自动写入 `thesis_template/generated/`。

在公平性扫描之后，后续 PPPM 基线统一采用 16^3 网格；RBSOG 默认采用 $P = 100$ 、 $M = 12$ 、短程截断半径 1.2。需要强调的是， $P = 100$ 是 RBSOG 候选参数中的折中推荐，不表示其已经优于 PPPM。

3.2 PPPM 网格公平性扫描

时间-误差折中

表3.1 给出了 PPPM 网格扫描结果。随着网格从 16^3 增大到 64^3 ，PPPM 单步时间明显上升；力误差和压力误差并非严格单调下降，说明当前简化 PPPM 实现和代理体系下，网格加密并不自动等价于所有误差指标同步改善。按本文定义的公平性评分， 16^3 被选为后续主基准网格，其相对力 RMSE 为 2.118，压力相对误差为 0.352。

表 3.1 PPPM 网格公平性扫描结果

Grid	Step Time (s)	Pressure Var	Force RMSE	Pressure Error	Fairness Score	Recommended
16^3	0.000783	9.251e-04	2.118	0.352	0.152	yes
24^3	0.001494	4.769e-04	2.256	0.351	0.274	no
32^3	0.003883	3.250e-04	2.747	0.438	0.710	no
48^3	0.016525	3.378e-04	1.946	0.262	0.313	no
64^3	0.036335	2.306e-04	2.086	0.339	0.718	no

该结果带来一个重要限定：本文后续结论评价的是当前原型与当前公平性选择下的相对表现，而不是成熟 PME/PPPM 生产实现的上限。换言之，即使在较粗网格 PPPM 基线下，RBSOG 仍需接受性能和统计稳定性的实测检验。

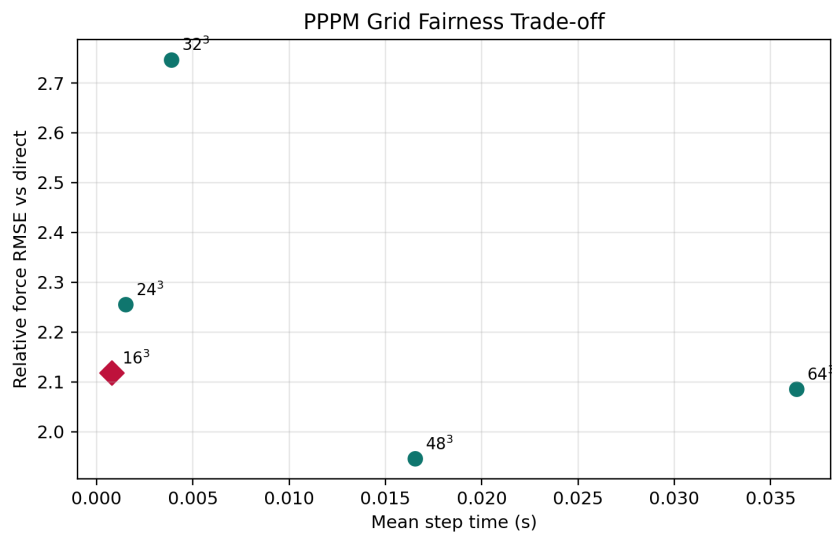


图 3.1 PPPM 网格时间-误差折中图。红色点为后续主基准采用的公平性推荐网格。

3.3 10 种子主基准测试

聚合结果

表3.2 汇总了 10 个随机种子的主基准结果。PPPM 平均每步时间为 0.000699 秒，RBSOG 为 0.025070 秒，RBSOG 约为 PPPM 的 35.868 倍。压力方差方面，RBSOG 均值为 PPPM 的 2.205 倍，说明在当前默认参数下，RBSOG 没有体现出压力波动优势。

表 3.2 10 种子主基准聚合结果

Solver	Mean Step Time (s)	Pressure Variance	Temp Mean	Area Drift	Thickness Drift
pppm	0.000699	8.412e-04	1.0287	-0.00708	-0.69542
rbsog	0.025070	1.855e-03	1.1981	-0.00659	2.72956

从结构代理量看，RBSOG 的面积/脂质漂移率与 PPPM 同阶，但膜厚漂移为正且幅度更大。这说明 RBSOG 不能被简单描述为“结构更稳定”。当前结果更支持如下判断：RBSOG 在部分横向结构量上与 PPPM 接近，但在法向膜厚代理量上引入了更明显的非稳态趋势。

运行级统计与效应量

表3.3 给出了运行级均值、标准差和 95% 置信区间。PPPM 步时间置信区间半宽为 0.000045 秒，RBSOG 为 0.000695 秒。由于两者均值相差远大于区间宽度，步时间差距不是随机波动造成的。



表 3.3 10 种子运行级统计摘要

Solver	n	Step Time Mean±Std	CI95 (Step)	Pressure Var Mean±Std
pppm	10	0.000699 ± 0.000063	0.000045	8.412e-04 ± 8.727e-05
rbso	10	0.025070 ± 0.000972	0.000695	1.855e-03 ± 3.552e-03

表3.4 进一步给出 Cohen’s d 效应量。步时间指标的效应量达到 35.391，属于极大效应；压力方差的效应量为 0.404，属于中等效应，但压力方差标准差很大，表明 RBSOG 在不同随机种子下存在明显离散性。

表 3.4 PPPM 与 RBSOG 主基准效应量分析

Metric	PPPM Mean±Std	RBSOG Mean±Std	Cohen’s d	Relative Change
Step time	0.000699 ± 0.000063	0.025070 ± 0.000972	35.391	3486.85%
Pressure variance	8.412e-04 ± 8.727e-05	1.855e-03 ± 3.552e-03	0.404	120.51%

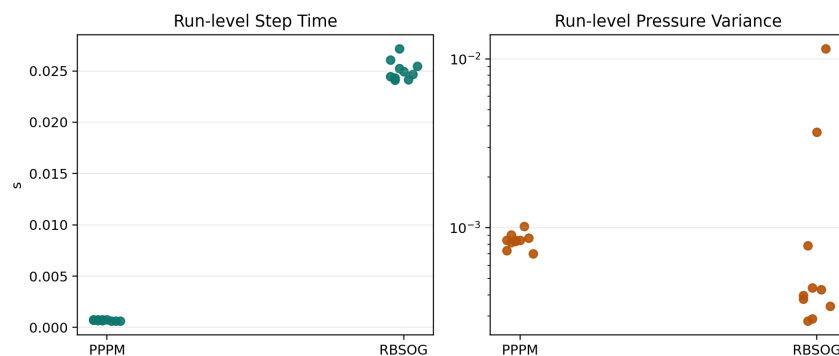


图 3.2 主基准运行级散点图。左图为步时间，右图为压力方差。

图3.2 显示，RBSOG 步时间分布相对集中但整体远高于 PPPM；压力方差则存在长尾种子。这一现象提示后续优化不仅要降低平均时间，也要控制随机采样带来的跨种子不稳定性。

3.4 RBSOG 批量大小扫描

Pareto 前沿与推荐点

表3.5 展示 $P \in \{50, 100, 200, 400\}$ 的批量扫描结果。所有候选点都在 Pareto 前沿上：小批量更快但压力方差更高，大批量降低压力方差但显著增加运行时间。在时间权重 0.667、方差权重 0.333 的设置下，自动推荐 $P = 100$ 。



表 3.5 RBSOG 批量大小扫描结果

P	Step Time (s)	Speedup vs PPPM	Variance Ratio	Utopia Score	Pareto	Recommended
50	0.014358	0.0533	0.93836	0.57735	yes	no
100	0.029636	0.0258	0.56559	0.21418	yes	yes
200	0.086335	0.0089	0.44569	0.61731	yes	no
400	0.109746	0.0070	0.41052	0.81650	yes	no

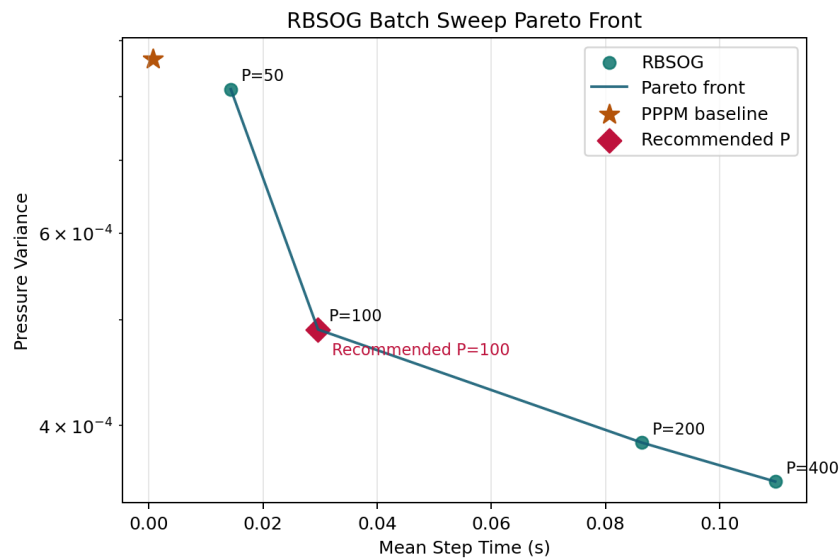


图 3.3 RBSOG 批量大小 Pareto 前沿。推荐点为 $P = 100$ 。



推荐点相对 PPPM 的速度比为 0.026，压力方差比为 0.5656。也就是说， $P = 100$ 的意义是“在 RBSOG 内部提供较合理折中”，不是“相对 PPPM 已经达到加速”。

敏感性趋势

图3.4 显示，随着 P 增大，RBSOG 步时间基本上升，而压力方差比下降。这一趋势符合随机批量方法的基本预期：更多样本能降低估计噪声，但会直接增加每步采样和长程累积成本。

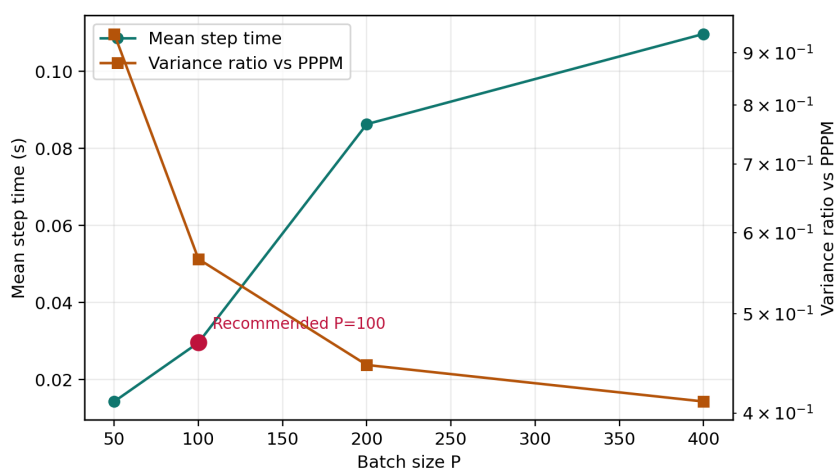


图 3.4 批量大小敏感性图。左轴为平均步时间，右轴为相对 PPPM 压力方差比。

3.5 Direct Reference 精度验证

批量大小对力误差的影响

表3.6 比较了不同体系规模下的相对力 RMSE。总体上，RBSOG 力误差随 P 增大而下降，尤其在 $N = 320$ 和 $N = 640$ 上趋势较清晰。例如 $N = 640$ 时， $P = 25$ 的力 RMSE 为 1.762，而 $P = 400$ 降至 0.942。这说明批量大小确实是控制随机估计误差的重要参数。

值得注意的是，公平性选择出的 PPPM 16^3 网格在 direct reference 下并非高精度配置，其力 RMSE 也较高。因此，本文不能把当前 PPPM 精度结果外推为成熟 PPPM 方法的真实精度上限；它只表示当前公平性成本下的原型基线。

SOG 项数对误差的影响

表3.7 展示了 M 对力误差的影响。结果并非“ M 越大越好”：在当前拟合区间与非负权重裁剪策略下， $M = 6$ 反而在三组体系上给出最低或接近最低的力



表 3.6 Direct reference 下不同批量大小的相对力 RMSE

System	PPPM	P=25	P=50	P=100	P=200	P=400
N160	2.070	1.437	1.261	1.153	1.113	1.092
N320	1.495	1.204	0.978	0.832	0.797	0.750
N640	1.726	1.762	1.430	1.157	1.020	0.942

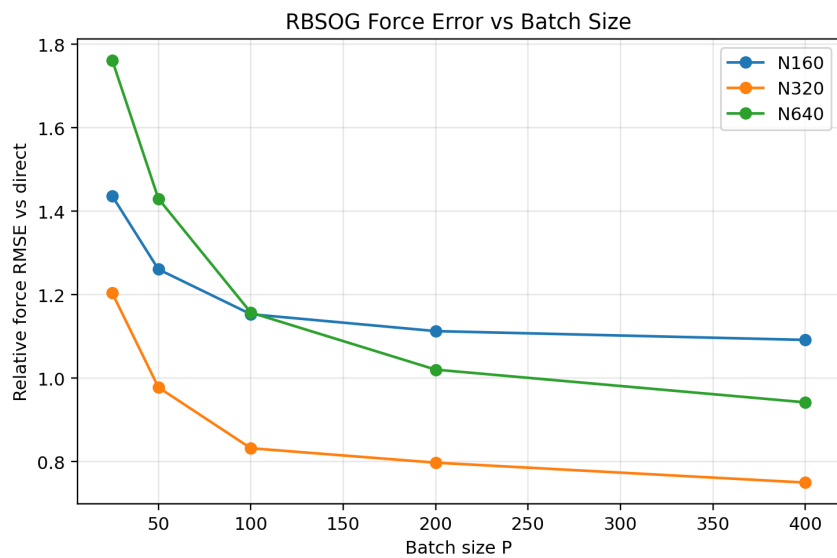


图 3.5 RBSOG 相对力 RMSE 随批量大小变化的趋势。



RMSE，而 $M = 18$ 和 $M = 24$ 会显著放大误差。这个现象说明当前 SOG 核拟合仍然是 RBSOG 误差来源中的关键环节，简单增加高斯项数不能替代稳定的核参数优化。

表 3.7 Direct reference 下不同 SOG 项数的相对力 RMSE

System	PPPM	M=6	M=12	M=18	M=24
N160	2.070	0.900	1.169	3.136	9.834
N320	1.495	0.714	0.880	2.163	6.802
N640	1.726	1.038	1.151	2.584	7.688

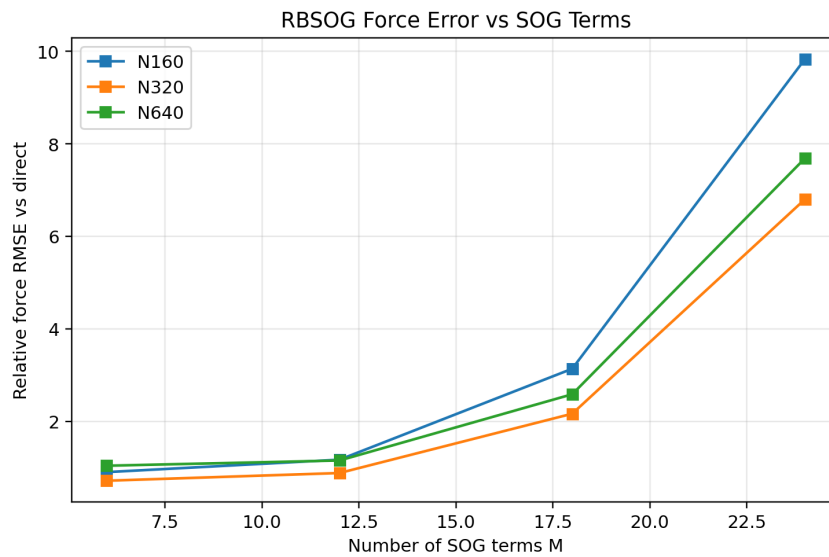


图 3.6 RBSOG 相对力 RMSE 随 SOG 项数变化的趋势。

3.6 规模扫描实验

平均步时间随规模变化

表3.8 和图3.7 给出了 $N = 320, 640, 960, 1280$ 的规模扫描结果。RBSOG 在所有规模上都慢于 PPPM，且慢速比从 21.982 扩大到 63.019。这说明在当前串行 Python/JIT 混合实现下，RBSOG 尚未表现出更好的经验扩展性。



表 3.8 规模扫描下的平均步时间与 RBSOG 相对 PPPM 慢速比

N	PPPM Step (s)	RBSOG Step (s)	Slowdown
320	0.000518	0.011384	21.982
640	0.000654	0.021538	32.956
960	0.000715	0.034060	47.666
1280	0.000774	0.048771	63.019

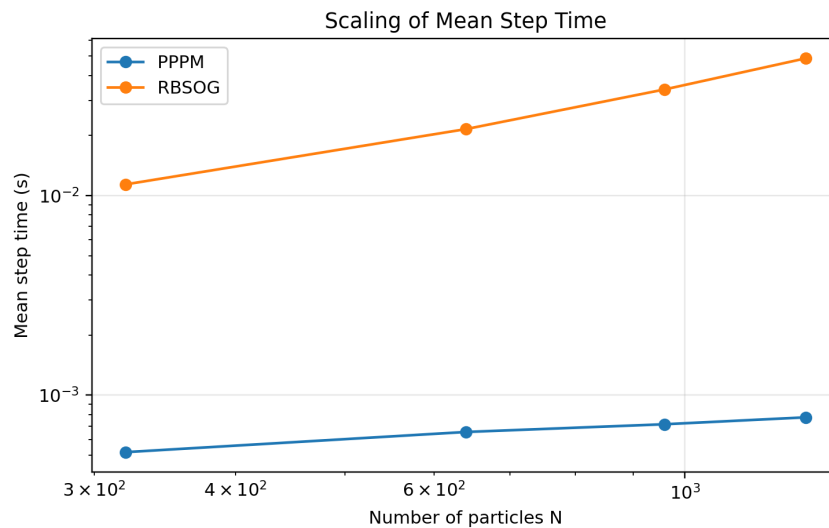


图 3.7 PPPM 与 RBSOG 平均步时间随粒子数变化的 log-log 曲线。



经验复杂度斜率

表3.9 给出对数-对数拟合结果。PPPM 经验斜率为 0.288，RBSOG 为 1.042。由于当前规模范围有限，这些斜率不能被解释为理论复杂度证明，但它们清楚表明：在本文可运行规模内，RBSOG 的时间增长更接近线性，而 PPPM 在该简化网格范围内增长更慢。

表 3.9 规模扫描 log-log 经验斜率

Solver	Empirical Slope	R^2
PPPM	0.288	0.992
RBSOG	1.042	0.995

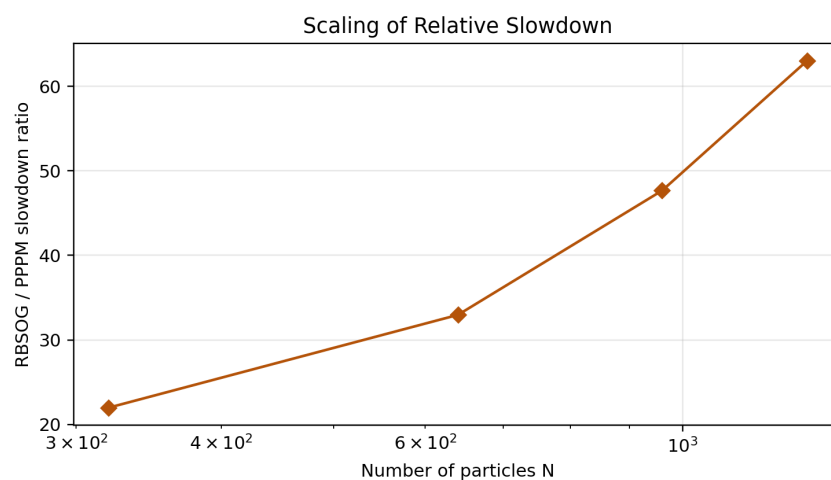


图 3.8 RBSOG 相对 PPPM 慢速比随粒子数变化。

3.7 长轨迹稳定性实验

5000 步统计摘要

表3.10 汇总了 5 个随机种子、5000 步长轨迹的结果。RBSOG 长轨迹步时间约为 PPPM 的 59.474 倍；压力方差明显大于 PPPM，但面积/脂质和膜厚代理量的标准差、漂移率反而低于 PPPM。这个结果与短轨迹主基准形成互补：RBSOG 在压力统计上仍不稳定，但部分结构代理量在长时间窗口中并未更差。

长轨迹结果提示，结构代理量和压力方差不能相互替代。压力是对力误差和采样噪声更敏感的全局统计量，而面积/脂质和膜厚代理量在 Berendsen 控制与简



表 3.10 5000 步长轨迹稳定性摘要

Solver	Step Time (s)	Pressure Var	Area Std	Thickness Std	Area Drift	Thickness Drift
PPPM	0.000627	2.719e-04	0.0202	1.7584	-0.0069	0.5116
RBSOG	0.037290	1.314e+00	0.0079	1.1057	-0.0013	0.1312

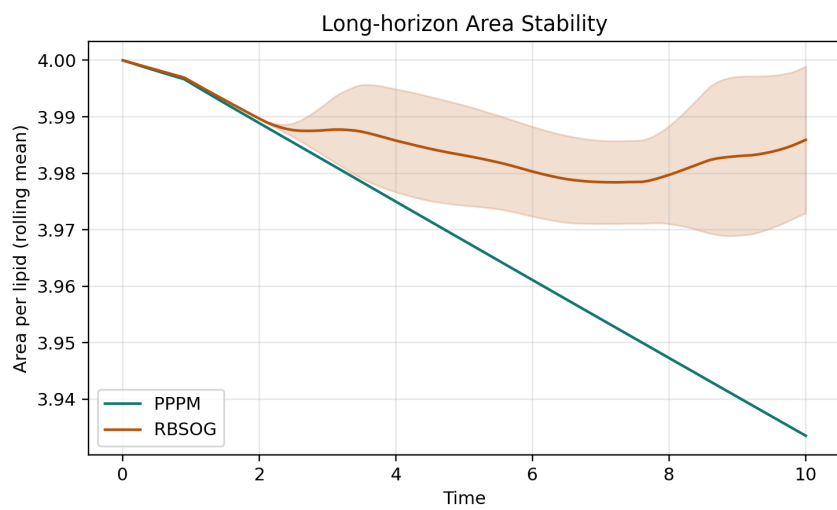


图 3.9 长轨迹面积/脂质滚动均值。阴影表示跨种子标准差。

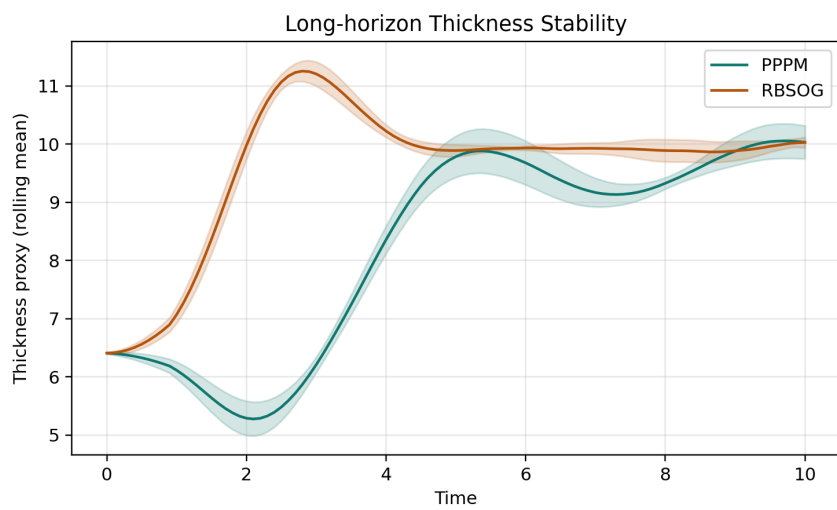


图 3.10 长轨迹膜厚代理量滚动均值。阴影表示跨种子标准差。

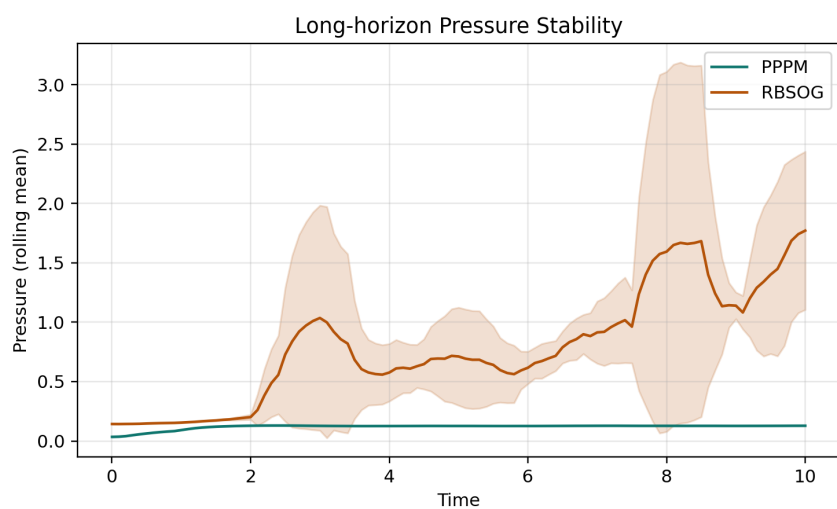


图 3.11 长轨迹压力滚动均值。RBSOG 存在明显更大的压力波动。

化代理体系下可能表现出不同的平滑特征。因此，本研究不把任何单一指标作为“稳定性胜负”的唯一依据。



3.8 RBSOG 参数消融与 Profiling

参数消融

表3.11 给出了 RBSOG 参数消融中的三个代表性结论：最佳力误差配置出现在 $P = 400, M = 6$ ；最快运行配置为 $P = 25, r_c = 1.6$ ；最低压力方差配置为 $P = 50, r_c = 1.6$ 。三者并不重合，说明 RBSOG 不存在一个在当前扫描空间内同时支配速度、方差与力误差的单一参数组。

表 3.11 RBSOG 参数消融摘要

Focus	Config	Step Time (s)	Pressure Var	Force RMSE
Best accuracy	$P=400, M=6$	–	–	0.623
Fastest runtime	$P=25, r_c=1.6$	0.006494	$1.163\text{e-}05$	–
Lowest variance	$P=50, r_c=1.6$	0.011581	$4.416\text{e-}06$	–

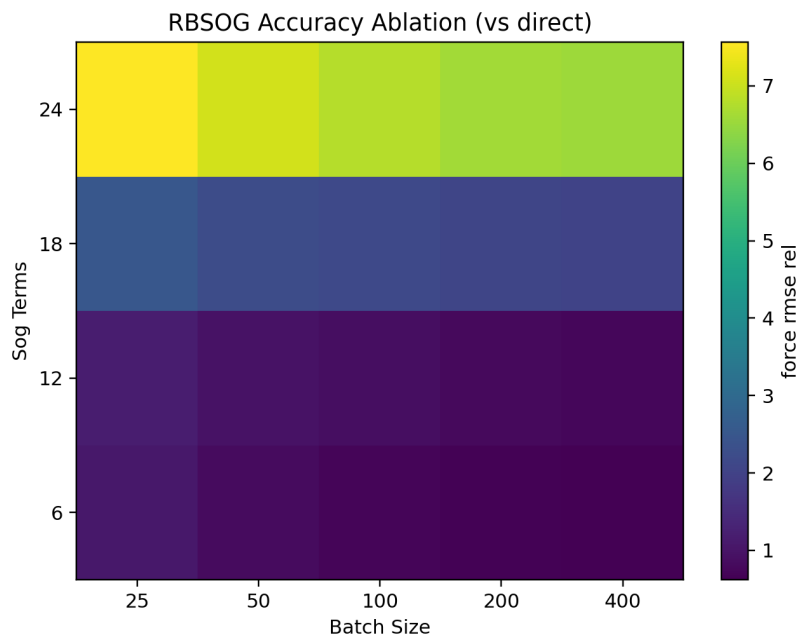


图 3.12 RBSOG 力误差消融热力图。

Profiling 分解

表3.12 和图3.14 给出 profiling 结果。PPPM 耗时主要集中在 FFT 求解，占 79.9%；RBSOG 耗时则主要集中在随机采样、长程核计算和样本过滤三部分，分别约占 38.6%、35.2% 和 15.9%。

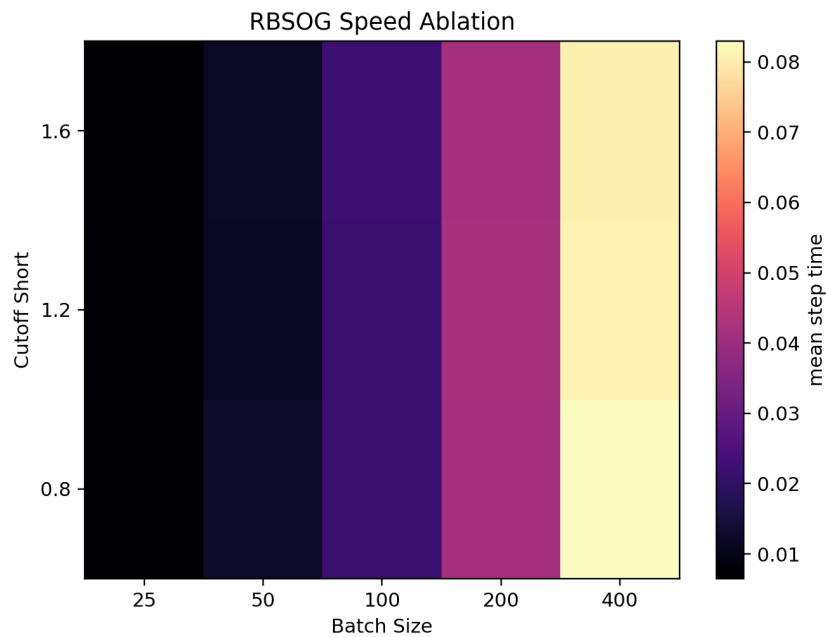


图 3.13 RBSOG 运行时间消融热力图。

表 3.12 PPPM 与 RBSOG 组件耗时占比

Solver	Component	Mean Time (s)	Share (%)
PPPM	pppm_fft_solve_time	0.000380	79.89
PPPM	pppm_field_interp_time	0.000050	10.48
PPPM	pppm_charge_deposition_time	0.000041	8.66
RBSOG	rbso_g_sampling_time	0.008813	38.63
RBSOG	rbso_g_long_range_time	0.008025	35.17
RBSOG	rbso_g_filter_time	0.003621	15.87
RBSOG	rbso_g_short_range_time	0.000737	3.23
RBSOG	neighbor_pair_build_time	0.000698	3.06
RBSOG	neighbor_short_range_time	0.000030	0.13
RBSOG	rbso_g_proposal_time	0.000013	0.06

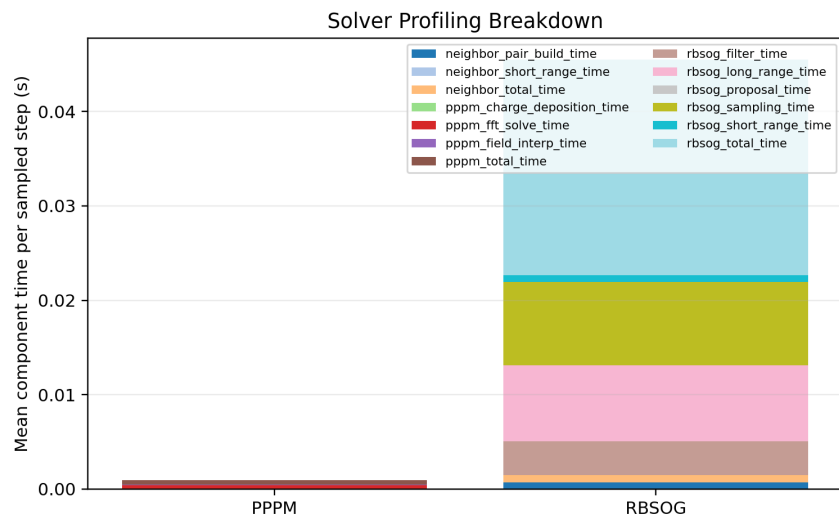


图 3.14 求解器组件耗时堆叠图。

该结果说明，当前 RBSOG 慢并非主要由短程近邻表造成，而是由随机样本生成、过滤和长程累积主导。因此，后续优化应优先减少长程采样路径的数据搬运、降低无效样本过滤成本，并把长程核进一步向连续内存和并行执行友好的形式改写。

JIT 消融

表3.13 和图3.15 展示 JIT 消融结果。开启 JIT 后，RBSOG 平均步时间从 0.105101 秒降至 0.024610 秒，获得 4.271 倍加速；压力方差比保持为 1.0000。这说明 JIT 主要改变实现效率，而没有改变统计性质。

表 3.13 RBSOG JIT 消融结果

Mode	Step Time (s)	Pressure Variance	Relative Speed	Variance Ratio
JIT on	0.024610	1.344e-04	4.271x	1.0000
JIT off	0.105101	1.344e-04	1.000x	1.0000

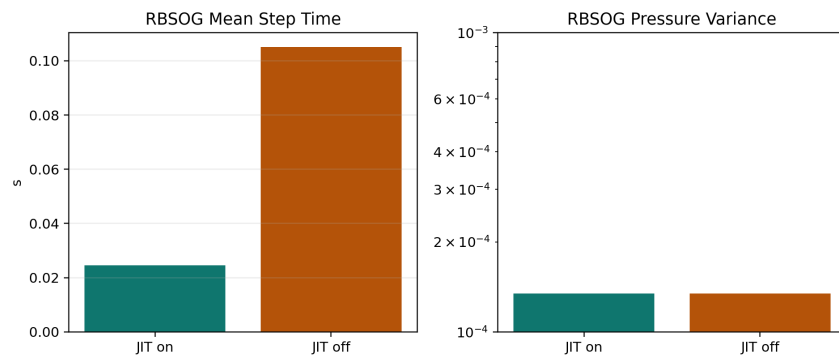


图 3.15 RBSOG JIT 开启/关闭下的步时间与压力方差对比。

3.9 稳健统计分析

表3.14 给出了 bootstrap 比例区间和置换检验结果。步时间比例为 35.868，95% bootstrap 区间为 [33.914, 38.062]，且置换检验 p 值接近 0，说明 RBSOG 较慢这一结论高度稳健。压力方差比例为 2.205，但 95% 区间为 [0.451, 5.119]，跨过 1，且均值差置换检验不显著。这意味着“RBSOG 平均压力方差更高”在当前样本中成立，但其统计不确定性显著大于步时间结论。

表 3.14 主基准稳健统计分析

Metric	Ratio	95% CI	Permutation p	Dispersion p
step_time	35.868	[33.914, 38.062]	0.0000	0.0000
pressure_variance	2.205	[0.451, 5.119]	0.4806	0.0893

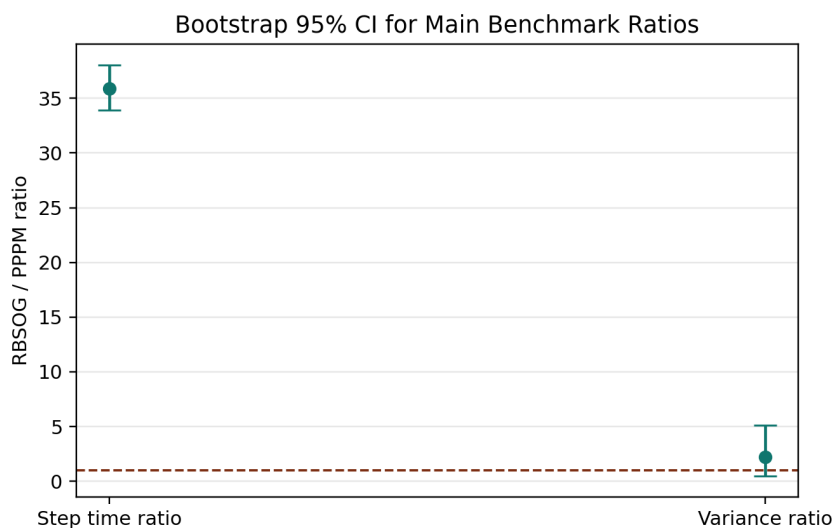


图 3.16 RBSOG/PPPM 均值比例的 bootstrap 95% 置信区间。

3.10 本章小结

基于完整实验套件，可以得到以下结论：

(1) 在公平性选择后的 16^3 PPPM 基线下，RBSOG 主基准步时间仍约慢 35.868 倍，且这一结论在 bootstrap 和置换检验下非常稳健。

(2) RBSOG 压力方差均值高于 PPPM，但跨种子波动较大，统计不确定性明显强于步时间结论；因此只能说当前默认参数下未体现压力稳定性优势，而不应过度外推。

(3) direct reference 实验表明，增大 P 往往能降低 RBSOG 力误差，而 SOG 项数并非越大越好；当前核拟合策略需要进一步优化。

(4) 规模扫描显示，在 $N = 320$ 到 $N = 1280$ 范围内，RBSOG 相对 PPPM 的慢速比随规模扩大而增加，当前实现尚未体现可扩展性优势。

(5) Profiling 表明 RBSOG 主要瓶颈位于随机采样、长程核计算和样本过滤，而非短程近邻表；JIT 可带来 4.271 倍加速，但仍不足以弥合与 PPPM 的差距。

综合来看，当前实验更支持“RBSOG 路线可实现、误差可调、瓶颈可定位”的原型验证结论，而不支持“当前实现已经优于 PPPM”的性能结论。



第四章 讨论

4.1 主要结果的含义

RBSOG 当前尚未体现性能优势

本文最清晰的结果是：在当前纯 Python/JIT 混合原型和本文可运行规模内，RBSOG 显著慢于 PPPM。主基准中，RBSOG 平均步时间约为 PPPM 的 35.868 倍；规模扫描中，慢速比从 $N = 320$ 时的 21.982 扩大到 $N = 1280$ 时的 63.019。稳健统计也支持这一点：步时间 bootstrap 比例区间为 $[33.914, 38.062]$ ，没有接近 1。

这一结果并不否定 RBSOG 路线的长期动机。RBSOG 希望减少对全局 FFT 通信路径的依赖，而当前实验是在单机串行原型上完成的，并未进入其潜在优势最可能体现的大规模并行环境。然而，从毕业论文实证角度看，当前数据必须被如实表述为：RBSOG 在现有实现下还没有转化为实测性能优势。

压力方差结论比步时间结论更不确定

压力方差均值上，RBSOG 高于 PPPM，主基准比例为 2.205。但 bootstrap 区间为 $[0.451, 5.119]$ ，跨过 1，置换检验也不显著。这说明压力方差结论不能像步时间结论那样强。更准确的表述是：当前默认参数下，RBSOG 没有表现出稳定的压力方差优势，并且若干种子存在明显长尾波动。

这种差异符合随机批量方法的基本特征。步时间主要由实现路径和样本数量决定，跨种子变化较小；压力方差则同时受采样噪声、初始条件、温控压控反馈和有限轨迹长度影响，因此更容易出现高不确定性。后续如果要对压力稳定性做更强结论，需要更长轨迹、更大种子数和更丰富的统计检验。

结构代理量与压力方差并不等价

长轨迹实验给出一个值得注意的现象：RBSOG 压力方差显著高于 PPPM，但面积/脂质和膜厚代理量的标准差、漂移率在 5000 步统计下并不更差。这说明结构代理量和压力方差不能互相替代。压力是对瞬时力、virial 和采样噪声更敏感的全局量；面积/脂质和膜厚代理量则在 Berendsen 控制和简化代理体系下更平滑。

因此，本文不把“压力方差更高”直接等同于“结构一定更差”，也不把“结构代理量更平滑”直接等同于“算法更稳定”。更稳妥的结论是：当前 RBSOG 的



统计压力行为仍需控制，而结构代理量没有给出单向否定或单向肯定的证据。

4.2 误差来源与参数敏感性

批量大小是有效但昂贵的误差控制旋钮

direct reference 实验显示，增大批量大小 P 通常能降低相对力 RMSE。例如在 $N = 640$ 体系上， $P = 25$ 的力 RMSE 为 1.762， $P = 400$ 降至 0.942。这说明随机批量采样的误差确实可以通过增加样本量缓解。

问题在于，这种缓解非常昂贵。批量扫描中， $P = 400$ 的压力方差最低，但步时间最高； $P = 50$ 最快，但方差更高。 $P = 100$ 是当前加权目标下的折中点，而不是全局支配点。这种结果说明 RBSOG 目前主要处在“可调但代价高”的阶段。

SOG 项数不是单调收益参数

另一个重要发现是，增加 SOG 项数 M 并不一定降低误差。在当前实现中， $M = 6$ 往往优于 $M = 12, 18, 24$ 。这说明当前高斯和核参数生成、最小二乘拟合区间、非负权重裁剪等细节对误差有很大影响。

从方法学角度看，这一点很关键。如果把 RBSOG 的误差简单归因于“随机批量采样噪声”，就会忽略 SOG 核拟合本身的误差贡献。后续工作应把误差分解为至少两部分：一是固定核下的随机采样误差，二是不同 SOG 核参数化方式造成的系统误差。

PPPM 公平性基线的边界

本文通过网格公平性扫描选择 16^3 作为 PPPM 基线。这有助于避免把过高成本的 PPPM 配置直接拿来与 RBSOG 比较，但也带来一个限制：该 PPPM 配置不是高精度 PME/PPPM 生产参数。direct reference 结果显示，公平性选择下的 PPPM 力 RMSE 也较高。

因此，本文所有“RBSOG 与 PPPM”的结论都应理解为当前原型公平配置下的比较，而不是对成熟软件中高精度 PPPM 的最终评价。这个限定不会削弱“RBSOG 当前更慢”的结论，反而使其更严格：即便在较低成本 PPPM 基线下，RBSOG 仍未取得性能优势。



4.3 Profiling 揭示的优化方向

当前瓶颈集中在长程采样路径

Profiling 结果显示, PPPM 耗时主要集中在 FFT 求解, 占 79.9%; RBSOG 耗时主要集中在随机采样、长程核计算和样本过滤, 三者合计约占 $38.6 + 35.2 + 15.9$ 的量级。短程近邻表不是主要瓶颈。

这意味着后续优化不应优先围绕短程 pair list 做小修小补, 而应集中处理长程采样路径: 减少随机索引生成开销, 避免生成后再大量过滤无效样本, 改进内存布局, 并将长程核改写为更适合向量化、并行化或 GPU 执行的数据流。

JIT 说明实现层仍有明显空间

JIT 消融显示, 开启 JIT 带来 4.271 倍加速, 而压力方差不变。这说明当前 RBSOG 慢速中确实存在大量实现层开销。与此同时, 即便开启 JIT, RBSOG 仍明显慢于 PPPM, 因此 JIT 只能证明“还有工程优化空间”, 不能证明“工程优化后必然超过 PPPM”。

更现实的路线是分阶段推进: 先把采样和过滤路径降到合理成本, 再比较 CPU 多线程或 GPU 版本中的相对扩展性, 最后才讨论是否能够在大规模并行场景中规避 FFT 通信瓶颈。

4.4 与相关工作的关系

传统 Ewald、PME 和 PPPM 方法通过实空间与倒易空间分解, 把长程静电计算复杂度降到适合大规模 MD 的水平^[8,9]。它们的短板主要在并行环境下的全局通信, 而不是单机小规模运行效率。本文结果也符合这一点: 在当前单机原型规模内, PPPM 仍然非常快。

随机批量方法的理论吸引力在于用随机估计降低每步交互规模^[10]。本文实现的 RBSOG 继承了这一思想, 但实验表明, 随机估计的计算收益只有在采样路径足够高效时才可能显现。否则, 随机索引生成、样本过滤和无偏权重修正本身会吞噬理论节省。

高斯和分解提供了处理奇异核的一种光滑近似方式^[11]。本文的 direct reference 实验说明, 这条路径可以运行并能被系统评测, 但当前核拟合策略还不够稳定。特别是 M 增大导致误差变差这一现象, 说明 SOG 分解在分子动力学静电场景中的参数化仍需专门研究。



4.5 局限性

本文仍有若干局限。

首先，体系是膜启发代理模型，不是全原子 DPPC 或真实细胞环境。因此，结论主要是算法工程层面的，不能直接外推为生物物理结论。

其次，当前实现仍是 Python 研究原型。尽管包含 Numba 加速，但与成熟 C++/CUDA 分子动力学引擎相比仍有明显差距。因此，本文比较的是原型实现而非生产级实现。

再次，规模扫描最大到 $N = 1280$ ，尚不足以检验 RBSOG 在超大规模并行环境中的低通信潜力。当前实验只说明它在可运行单机规模内尚未体现优势。

最后，direct reference 精度实验虽然覆盖了 $N = 160, 320, 640$ ，但仍只使用有限快照和有限参数组合。未来需要更系统地区分 SOG 核误差、随机采样误差、短程截断误差和积分反馈误差。

4.6 后续工作

基于本文结果，后续工作应优先从以下方向展开：

- (1) 重构长程采样路径，避免生成大量随后被过滤的无效样本；
- (2) 重新设计 SOG 核拟合流程，比较不同拟合区间、权重约束和误差目标；
- (3) 增加更大规模和更长轨迹实验，区分短期压力噪声与长期结构趋势；
- (4) 开发多线程或 GPU 版本，真正测试 RBSOG 规避 FFT 通信的潜在优势；
- (5) 迁移到更真实的粗粒化或全原子体系，同时保持当前自动化实验与论文资产链路。

总体而言，当前工作完成了从方法想法到可复现实验体系的闭环，也明确指出了路线的真实困难。这比只报告单次有利结果更有价值，因为它为后续优化提供了可验证的靶点。



第五章 结论

5.1 主要工作总结

本文围绕全细胞分子动力学模拟中的长程静电求解问题，构建了一个面向膜体系代理模型的 RBSOG-MD 研究原型，并实现了 direct reference、PPPM 基线和 RBSOG 三类求解器。研究重点不是证明当前原型已经优于成熟 PPPM，而是建立一套可复现的实验流程，真实评估 RBSOG 在性能、统计方差、精度、规模趋势和实现瓶颈上的当前状态。

本文完成的主要工作包括：

(1) 实现了基于高斯和分解与随机批量采样的 RBSOG 长程静电求解框架，并与 PPPM 和 direct solver 放在同一套模拟接口下比较。

(2) 构建了自动化实验套件，覆盖 PPPM 网格公平性扫描、10 种子主基准、RBSOG 批量扫描、direct reference 精度验证、规模扫描、5000 步长轨迹稳定性、参数消融、profiling 和 bootstrap/置换检验。

(3) 建立了从实验输出到论文图表的自动生成链路，使 results/thesis_suite/ 中的 CSV、JSON 和 PNG 结果能够自动同步到 thesis_template/generated/，降低手工整理数据造成的不一致风险。

(4) 通过真实实验明确了当前 RBSOG 原型的边界：方法可实现、误差可调、瓶颈可定位，但在当前实现和当前规模下尚未优于 PPPM。

5.2 核心实验结论

本文的核心结论可以概括为三层。

第一，**RBSOG 当前实现尚未在性能上超过 PPPM**。在 10 种子主基准中，RBSOG 平均每步时间约为 PPPM 的 35.868 倍；在规模扫描中， $N = 1280$ 时慢速比达到 63.019。bootstrap 比例区间 [33.914, 38.062] 也表明这一结论非常稳健。

第二，**RBSOG 默认参数下没有体现稳定的压力方差优势**。主基准中 RBSOG 压力方差均值约为 PPPM 的 2.205 倍，但 bootstrap 区间跨过 1，说明该指标不确定性较大。更准确的表述是：当前 RBSOG 压力统计存在明显种子敏感性，尚不能宣称比 PPPM 更稳定。

第三，**RBSOG 的误差与性能具有可调节性，但不同目标之间存在冲突**。增大批量大小 P 往往能降低 direct reference 力误差，却显著增加运行时间；SOG 项



数 M 也不是单调收益参数。参数消融显示，最佳力误差、最快运行时间和最低压力方差对应不同配置，说明当前方法仍需要更系统的参数化和优化。

5.3 创新点与意义

本文的创新点不在于给出一个已经打败 PPPM 的最终算法，而在于完成了一个可追溯、可运行、可证伪的原型研究闭环。

首先，本文将高斯和分解与随机批量采样结合到长程静电原型求解中，给出了一条绕开全局 FFT 通信路径的具体实现路线。

其次，本文没有只报告单一性能数字，而是把性能、压力方差、direct reference 误差、规模趋势、长轨迹结构代理量、JIT 消融和 profiling 放入同一分析框架。这使得“为什么慢”“哪里不稳定”“哪些参数有效”能够被同时讨论。

最后，本文把实验和论文资产生成自动化。对后续研究而言，只要重新运行实验套件，就能得到新的表格、图像和宏定义，从而持续更新论文证据链。这种 workflow 本身也是科研工程实践的重要组成部分。

5.4 最终判断

基于本文全部实验，可以给出一个清晰而克制的最终判断：**RBSOG 在当前研究原型中尚未在速度和压力统计稳定性上超过 PPPM，但已经形成了可实现、可评测、可优化的低通信长程静电算法原型。**

这一判断包含两个方面。一方面，当前 RBSOG 距离实用性能仍有明显差距，尤其是随机采样、样本过滤和长程核计算构成主要瓶颈。另一方面，JIT 消融带来 4.271 倍加速，profiling 明确指出主要耗时组件，direct reference 实验也显示误差可以随参数变化而调节。因此，RBSOG 路线仍具有继续优化和迁移到并行实现中的研究价值。

未来工作的重点不应是简单增加实验次数来寻找个别有利样本，而应围绕本文定位出的核心问题展开：改进 SOG 核拟合，降低长程采样路径成本，减少无效样本过滤，扩展到多线程或 GPU 环境，并在更真实的分子体系上复核这些结论。只有完成这些步骤之后，才能进一步判断 RBSOG 是否能够在大规模分子动力学模拟中真正体现其低通信潜力。



附录 A 实现补充与复现实验说明

A.1 真实实现说明

附录不再给出与仓库实现不一致的示例代码, 而是摘录当前仓库中的真实核心实现, 并用少量注释说明其与正文算法描述的对应关系。本文涉及的核心源码主要位于 `src/rbsog_md/forces/rbsog.py`、`src/rbsog_md/forces/neighbor.py`、`src/rbsog_md/forces/sog.py` 与 `src/rbsog_md/benchmark.py`。

当前实现有三个与正文结论直接相关的特征:

- (1) 短程库仑相互作用由 Verlet 近邻表求解器负责, 支持 Numba 加速;
- (2) 长程部分不是简单地“抽取一个固定批量内的所有粒子对”, 而是对粒子索引按重要性分布进行采样, 再使用重要性权重进行无偏校正;
- (3) SOG 核通过最小二乘拟合得到, 并在给定盒子尺寸下自动重建。

A.2 RBSOG 求解器真实核心代码节选

下面节选自 `src/rbsog_md/forces/rbsog.py`。这一实现展示了: 先计算短程结果, 再对长程部分进行重要性采样, 最后将两部分贡献合并。

```
1 @dataclass
2 class RBSOGConfig:
3     batch_size: int = 100
4     cutoff_short: float = 1.2
5     sog_terms: int = 12
6     r_min: float = 0.15
7     importance_exponent: float = 1.0
8     neighbor_skin: float = 0.3
9     neighbor_rebuild_interval: int = 10
10    use_numba: bool = True
11
12
13 class RBSOGSolver:
14     name = "rbsog"
15
16     def __init__(self, config: RBSOGConfig) -> None:
17         self.config = config
```



```
18         self.short_range_solver = NeighborListCoulombSolver(  
19             cutoff=config.cutoff_short,  
20             skin=config.neighbor_skin,  
21             rebuild_interval=config.neighbor_rebuild_interval,  
22             use_numba=config.use_numba,  
23         )  
24         self.long_range_jit_enabled = bool(config.use_numba and  
25         _NUMBA_AVAILABLE)  
26         self.kernel: SOGKernel | None = None  
27         self.kernel_box: np.ndarray | None = None  
28  
29     def _ensure_kernel(self, box: np.ndarray) -> None:  
30         if self.kernel is not None and self.kernel_box is not  
31         None and np.allclose(box, self.kernel_box):  
32             return  
33         r_max = 0.5 * float(np.min(box))  
34         self.kernel = fit_sog_kernel(  
35             r_min=self.config.r_min,  
36             r_max=r_max,  
37             n_terms=self.config.sog_terms,  
38         )  
39         self.kernel_box = box.copy()  
40  
41     def compute(self, system: ParticleSystem, rng: np.random.  
42     Generator | None = None) -> ForceResult:  
43         if rng is None:  
44             rng = np.random.default_rng()  
45  
46         self._ensure_kernel(system.box)  
47         assert self.kernel is not None  
48  
49         short_result = self.short_range_solver.compute(system)  
50         forces = short_result.forces.copy()  
51         potential = short_result.potential  
52         virial = short_result.virial  
53  
54         positions = system.positions
```



```
52     charges = system.charges
53     n_particles = system.n_particles
54     m_samples = max(self.config.batch_size * n_particles, 1)
55
56     q_abs = np.abs(charges) + 1e-6
57     proposal = q_abs**self.config.importance_exponent
58     proposal /= np.sum(proposal)
59
60     i_all = rng.choice(n_particles, size=m_samples, p=
proposal)
61     j_all = rng.choice(n_particles, size=m_samples, p=
proposal)
62     mask_not_self = i_all != j_all
63
64     i_idx = i_all[mask_not_self]
65     j_idx = j_all[mask_not_self]
66     displacement = minimum_image(positions[j_idx] - positions
[i_idx], system.box)
67     r_sq = np.einsum("ij,ij->i", displacement, displacement)
68
69     cutoff_sq = self.config.cutoff_short * self.config.
cutoff_short
70     r_min_sq = self.config.r_min * self.config.r_min
71     mask_long = (r_sq > cutoff_sq) & (r_sq > r_min_sq)
72
73     i_valid = i_idx[mask_long]
74     j_valid = j_idx[mask_long]
75     disp_valid = displacement[mask_long]
76     r_sq_valid = r_sq[mask_long]
77
78     pair_prob = proposal[i_valid] * proposal[j_valid]
79     sample_weights = 1.0 / (m_samples * pair_prob)
80
81     pref = self.kernel.force_prefactor(r_sq_valid)
82     qij = charges[i_valid] * charges[j_valid]
83     pair_force = (qij * pref)[: , None] * disp_valid
84
```



```
85         scaled_force = sample_weights[:, None] * pair_force
86         np.add.at(forces, i_valid, scaled_force)
87         np.add.at(forces, j_valid, -scaled_force)
88
89         pair_potential = qij * self.kernel.evaluate_from_r_sq(
90         r_sq_valid)
91         potential += float(np.sum(0.5 * sample_weights *
92         pair_potential))
93         virial += float(np.sum(0.5 * sample_weights * np.einsum("
94         ij,ij->i", disp_valid, pair_force)))
95
96         return ForceResult(forces=forces, potential=potential,
97         virial=virial)
```

Listing 1 RBSOG 求解器真实核心代码节选

这一节选说明，正文中“随机批量采样”的实现应理解为“基于粒子电荷幅值的成对重要性采样”，而不是简单的均匀批量抽样。

A.3 近邻表与短程求解真实实现节选

下面代码节选自 `src/rbsog_md/forces/neighbor.py`，展示了 Verlet 近邻表的重建判据和短程库仑求解流程。

```
1 @dataclass
2 class VerletNeighborList:
3     cutoff: float
4     skin: float = 0.3
5     rebuild_interval: int = 10
6
7     @property
8     def pair_cutoff(self) -> float:
9         return self.cutoff + self.skin
10
11     def _needs_rebuild(self, system: ParticleSystem) -> bool:
12         if self._pairs is None or self._ref_positions is None or
13         self._ref_box is None:
14             return True
15         if not np.allclose(system.box, self._ref_box):
```



```
15         return True
16     if self._steps_since_build >= self.rebuild_interval:
17         return True
18     if self.skin <= 0.0:
19         return False
20
21     displacement = minimum_image(system.positions - self._ref_positions, system.box)
22     max_disp_sq = float(np.max(np.einsum("ij,ij->i", displacement, displacement)))
23     return max_disp_sq > (0.5 * self.skin) ** 2
24
25
26 class NeighborListCoulombSolver:
27     def compute(self, system: ParticleSystem, rng: np.random.Generator | None = None) -> ForceResult:
28         del rng
29         pairs = self.neighbor_list.get_pairs(system)
30         if pairs.size == 0:
31             return ForceResult(forces=np.zeros_like(system.positions), potential=0.0, virial=0.0)
32
33         return self._compute_vectorized(
34             positions=system.positions,
35             charges=system.charges,
36             box=system.box,
37             pairs=pairs,
38             cutoff_sq=self.cutoff * self.cutoff,
39             softening_sq=self.softening * self.softening,
40         )
```

Listing 2 近邻表与短程求解真实代码节选

A.4 SOG 核拟合真实实现节选

下面代码节选自仓库中的 SOG 核拟合实现文件：

src/rbsog_md/forces/sog.py



其核心是：在给定 $[r_{\min}, r_{\max}]$ 区间内，对 $1/r$ 进行最小二乘拟合，得到用于长程近似的高斯核参数。

```
1 def fit_sog_kernel(  
2     r_min: float,  
3     r_max: float,  
4     n_terms: int = 12,  
5     n_samples: int = 4096,  
6 ) -> SOGKernel:  
7     alpha_min = 0.5 / (r_max * r_max)  
8     alpha_max = 8.0 / (r_min * r_min)  
9     alphas = np.geomspace(alpha_min, alpha_max, n_terms)  
10  
11     r = np.geomspace(r_min, r_max, n_samples)  
12     design = np.exp(-np.outer(r * r, alphas))  
13     target = 1.0 / r  
14  
15     weights, _, _, _ = np.linalg.lstsq(design, target, rcond=None  
16 )  
17     weights = np.clip(weights, 0.0, None)  
18  
19     return SOGKernel(alphas=alphas, weights=weights, r_min=r_min,  
20                       r_max=r_max)
```

Listing 3 SOG 核拟合真实代码节选

A.5 复现实验命令

本文最终采用的实验流程与仓库中的命令行入口一致。对应协议可见 docs/experiment_protocol.md，核心命令如下：

```
1 # PPPM vs RBSOG 主基准  
2 rbsog-md benchmark \  
3     --solvers ppm, rbsog \  
4     --steps 1000 \  
5     --sample-interval 20 \  
6     --seeds 3 \  
7     --n-lipids 128 \  
8     --n-solvent 512 \  

```



```
9  --batch-size 100 \  
10 --grid 32 32 32 \  
11 --out results/thesis_final_benchmark_main  
12  
13 # 批量大小扫描  
14 rbsog-md sweep-batch \  
15   --steps 1000 \  
16   --sample-interval 20 \  
17   --seeds 3 \  
18   --batch-sizes 50,100,200,400 \  
19   --objective-time-weight 2 \  
20   --objective-variance-weight 1 \  
21   --out results/thesis_final_batch_sweep  
22  
23 # JIT 消融  
24 rbsog-md benchmark \  
25   --solvers rbsog \  
26   --steps 400 \  
27   --sample-interval 10 \  
28   --seeds 2 \  
29   --batch-size 100 \  
30   --out results/thesis_ablation_jit_on  
31  
32 rbsog-md benchmark \  
33   --solvers rbsog \  
34   --steps 400 \  
35   --sample-interval 10 \  
36   --seeds 2 \  
37   --batch-size 100 \  
38   --disable-numba \  
39   --out results/thesis_ablation_jit_off
```

Listing 4 论文核心实验命令



A.6 附录小结

附录的目的不是重复展示与正文无关的大段模板代码，而是说明论文中对 RBSOG、近邻表、SOG 核拟合以及实验流程的描述都能在当前仓库真实实现中找到对应依据。这样可以确保正文、附录、源码三者之间保持一致。



参考文献

- [1] ELLIS R J. Macromolecular crowding: obvious but underappreciated[J]. Trends in Biochemical Sciences, 2001, 26(10): 597–604.
- [2] MCCAMMON J A, GELIN B R, KARPLUS M. Dynamics of folded proteins[J]. Nature, 1977, 267(5612): 585–590.
- [3] LENNER N, et al. Whole-cell simulations: challenges and applications[J]. Annual Review of Biophysics, 2023, 52: 1–25.
- [4] FEIG M, et al. Current progress and future directions in molecular dynamics simulations of cells [J]. Journal of Chemical Theory and Computation, 2023, 19(15): 5001–5022.
- [5] SPITZER J, et al. Molecular dynamics simulations of cells[J]. Current Opinion in Structural Biology, 2023, 80: 102570.
- [6] ZHAO G, et al. Mature hiv-1 capsid structure by cryo-electron microscopy and all-atom molecular dynamics[J]. Nature, 2013, 497(7451): 643–646.
- [7] CASALINO L, et al. All-atom molecular dynamics simulations of the complete sars-cov-2 virion [J]. ACS Central Science, 2020, 6(11): 1983–1996.
- [8] DARDEN T, YORK D, PEDERSEN L. Particle mesh ewald: An $n \log n$ method for ewald sums in large systems[J]. The Journal of Chemical Physics, 1993, 98(12): 10089–10092.
- [9] HOCKNEY R W, EASTWOOD J W. Computer simulation using particles[M]. [S.l.]: CRC Press, 1988.
- [10] JIN S, LI L, LIU J G. Random batch methods (rbm) for interacting particle systems[J]. Journal of Computational Physics, 2020, 400: 108877.
- [11] GUO X, HE Y, WANG L L. Sum-of-gaussians tensor neural networks for high-dimensional schrödinger equation[J]. arXiv preprint arXiv:2508.10454, 2025.



致 谢

首先,我要衷心感谢我的指导教师 XXX 教授。在整个毕业设计过程中,XXX 教授给予了我悉心的指导和宝贵的支持。从课题选定、方案设计到实验分析、论文撰写,XXX 教授都给予了我耐心的指导和建设性的意见。XXX 教授严谨的治学态度、渊博的学识和敏锐的学术洞察力,使我受益匪浅,也为我今后的学习和工作树立了榜样。

感谢上海科技大学信息科学与技术学院的各位老师,他们在我的本科学习期间给予了我许多帮助和指导。特别感谢 XXX 老师在分子动力学模拟方面的专业指导,以及 XXX 老师在高性能计算方面的技术支持。

感谢课题组的各位同学,他们在实验过程中给予了我许多帮助和支持。特别感谢 XXX 同学在代码实现方面的贡献,以及 XXX 同学在数据分析方面的协助。

感谢上海科技大学提供的良好学习环境和丰富的学术资源,使我能够顺利完成本科学业和毕业设计。

感谢开源社区的众多贡献者,特别是 LaTeX 模板的开发者们。他们的辛勤付出和非凡工作,使得我能够使用 LaTeX 高效地撰写学位论文。特别感谢 ucasthesis 模板的开发者,以及 gbt7714-bibtex-style 的开发者,他们的工作为我的论文撰写提供了极大的便利。

感谢我的家人,他们在我求学期间给予了我无条件的支持和鼓励。他们的理解和关爱,是我前进的动力。

最后,感谢所有在我求学期间给予我帮助和支持的人。你们的善意和帮助,我将永远铭记在心。